

Computer Vision - Exercise 4 Object Recognition

Ziyu Ma

2. Bag-of-words Classifier (60%)

2.1 Local Feature Extraction (20%)

2.1.1 Feature detection - feature points on a grid (5%)

```
def grid_points(img, nPointsX, nPointsY, border):  
    """  
    :param img: input gray img, numpy array, [h, w]  
    :param nPointsX: number of grids in x dimension  
    :param nPointsY: number of grids in y dimension  
    :param border: leave border pixels in each image dimension  
    :return: vPoints: 2D grid point coordinates, numpy array,  
    [nPointsX*nPointsY, 2]  
    """  
    # todo  
    h = img.shape[0]  
    w = img.shape[1]  
  
    x = np.linspace(border, w - border - 1, num = nPointsX, dtype=int)  
    y = np.linspace(border, h - border - 1, num = nPointsY, dtype=int)  
  
    vPoints = np.transpose([np.repeat(x, nPointsY), np.tile(y,  
nPointsX)])  
  
    return vPoints
```

For this part, I use `np.linspace()` to divide the `[border, w-border-1]` into `nPointsX` and `[border, h-border-1]` into `nPointsY` evenly spaced integer coordinates. Then `np.repeat()` (if we have `x` entries as `x1, x2, x3...`, then `np.repeat(x, nPointsY)` is `x1, x1...x1, x2, x2...x2, x3, x3...x3...` where each entry is repeated `nPointsY` times) and `np.tile()` (if we have `y` entries as `y1, y2, y3...`, then `np.tile(y, nPointsX)` is `y1, y2, y3..., y1, y2, y3...` where the whole array is repeated `nPointsX` times) to compose the `x` and `y` coordinates.

2.1.2 Feature description - histogram of oriented gradients (15%)

```
def descriptors_hog(img, vPoints, cellWidth, cellHeight):  
    nBins = 8  
    w = cellWidth  
    h = cellHeight  
  
    grad_x = cv2.Sobel(img, cv2.CV_16S, dx=1, dy=0, ksize=1)  
    grad_y = cv2.Sobel(img, cv2.CV_16S, dx=0, dy=1, ksize=1)  
  
    descriptors = [] # list of descriptors for the current image, each  
entry is one 128-d vector for a grid point  
    for i in range(len(vPoints)):  
        center_x = round(vPoints[i, 0])
```

```

center_y = round(vPoints[i, 1])

desc = []
for cell_y in range(-2, 2):
    for cell_x in range(-2, 2):
        start_y = center_y + (cell_y) * h
        end_y = center_y + (cell_y + 1) * h

        start_x = center_x + (cell_x) * w
        end_x = center_x + (cell_x + 1) * w

        # todo
        # compute the angles
        # compute the histogram

        id_x = np.arange(start_x, end_x)
        id_y = np.arange(start_y, end_y)

        angle = np.arctan2(grad_y[id_y, id_x], grad_x[id_y,
id_x]) % (2 * np.pi)

        #first separate into 16 bins, then assign the two bins
to the bin of closest direction among the eight directions.
        #e.g. for the direction of the x-coordinate, it
contains bin 0 and 15.
        idx_angle = (np.round(angle * 16 / (2 *
np.pi))).astype(np.intc)
        idx_angle[np.where(idx_angle == 16)] = 0
        idx_angle[np.where((idx_angle == 0))] = 0
        idx_angle[np.where((idx_angle == 15))] = 0
        idx_angle[np.where((idx_angle == 1))] = 1
        idx_angle[np.where((idx_angle == 2))] = 1
        idx_angle[np.where((idx_angle == 3))] = 2
        idx_angle[np.where((idx_angle == 4))] = 2
        idx_angle[np.where((idx_angle == 5))] = 3
        idx_angle[np.where((idx_angle == 6))] = 3
        idx_angle[np.where((idx_angle == 7))] = 4
        idx_angle[np.where((idx_angle == 8))] = 4
        idx_angle[np.where((idx_angle == 9))] = 5
        idx_angle[np.where((idx_angle == 10))] = 5
        idx_angle[np.where((idx_angle == 11))] = 6
        idx_angle[np.where((idx_angle == 12))] = 6
        idx_angle[np.where((idx_angle == 13))] = 7
        idx_angle[np.where((idx_angle == 14))] = 7

        hist, bin_edges = np.histogram(idx_angle,
bins=[0,1,2,3,4,5,6,7,8])

```

```

        desc = np.append(desc, hist)

    descriptors.append(desc)

    descriptors = np.asarray(descriptors) # [nPointsX*nPointsY, 128],
    descriptor for the current image (100 grid points)
    return descriptors

```

For this part, what we need to add in the code is to create an 8-bin histogram over the orientation of the gradient at each pixel for each cell (cellWidth*cellHeight pixels). First, I use np.arange() to get the indices and use them to get the gradient values. Those gradient values are then used to calculate the angles between x and y gradients. To assign the angles into 8-bins. I first divide the whole 360 degree space into 16 bins, then combine the two bins closest to one direction of the 8 bins. E.g. bin 0 and bin 15 are closest to the bin represented by direction of unit x. In the end, np.histogram() is used to create the histogram.

2.2 Codebook construction (15%)

```

    vPoints = grid_points(img, nPointsX, nPointsY, border)
    vFeatures.append(descriptors_hog(img, vPoints, cellWidth,
cellHeight))

```

For this part, I add the code to use grid_points() method to get the vPoints and then feed it to the descriptors_hog() to get the 128 dimension descriptor for each grid point. The descriptors are added to the vFeatures to be clustered.

2.3 Bag-of-words Vector Encoding (15%)

2.3.1 Bag-of-Words histogram (10%)

```

def bow_histogram(vFeatures, vCenters):
    """
    :param vFeatures: MxD matrix containing M feature vectors of dim. D
    :param vCenters: NxD matrix containing N cluster centers of dim. D
    :return: histo: N-dim. numpy vector containing the resulting BoW
activation histogram.
    """
    histo = None

    # todo
    N = vCenters.shape[0]
    Idx = findnn(vFeatures, vCenters)[0]
    histo = np.histogram(Idx, bins=N)[0]

    return histo

```

For this part, I use findnn() methods to find the closest cluster center for each of the vFeatures and then use np.histogram() to create the histogram.

2.3.2 Processing a directory with training examples (5%)

```

vPoints = grid_points(img, nPointsX, nPointsY, border)

```

```

        vFeatures = descriptors_hog(img, vPoints, cellWidth,
cellHeight)
        histo = bow_histogram(vFeatures, vCenters)
        vBoW.append(histo)

```

For this part, for each image, we first get the grid points, then the descriptors for each grid point, then the bag of words histogram. All images' bag of words histograms are then composed into vBoW.

2.4 Nearest Neighbor Classification (10%)

```

def bow_recognition_nearest(histogram, vBowPos, vBowNeg):
    """
    :param histogram: bag-of-words histogram of a test image, [1, k]
    :param vBowPos: bag-of-words histograms of positive training
images, [n_imgs, k]
    :param vBowNeg: bag-of-words histograms of negative training
images, [n_imgs, k]
    :return: sLabel: predicted result of the test image, 0(without
car)/1(with car)
    """

    # Find the nearest neighbor in the positive and negative sets and
decide based on this neighbor
    # todo
    DistPos = np.min(findnn(histogram, vBowPos)[1])
    DistNeg = np.min(findnn(histogram, vBowNeg)[1])

    if (DistPos < DistNeg):
        sLabel = 1
    else:
        sLabel = 0
    return sLabel

```

For this part, findnn() is used again to find the closest feature of vBowPos and vBowNeg from histogram. The distances are then compared to decide the label.

Result (Logging)

```

Anaconda Powershell Prompt (Miniconda)
(base) PS C:\Users\MZU> cd "D:\2023HS\Computer Vision\Object Recognition\exercise4_object_recognition_code"
(base) PS D:\2023HS\Computer Vision\Object Recognition\exercise4_object_recognition_code> conda activate ex4_cuda
(ex4_cuda) PS D:\2023HS\Computer Vision\Object Recognition\exercise4_object_recognition_code> python bow_main.py
creating codebook ...
100% 100/100 [00:13<00:00, 7.57it/s]
number of extracted features: 10000
clustering ...
D:\Miniconda\envs\ex4_cuda\lib\site-packages\sklearn\cluster\_kmeans.py:1416: FutureWarning: The default value of 'n_init'
will change from 10 to 'auto' in 1.4. Set the value of 'n_init' explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
creating bow histograms (pos) ...
100% 50/50 [00:06<00:00, 7.32it/s]
creating bow histograms (neg) ...
100% 50/50 [00:06<00:00, 7.77it/s]
creating bow histograms for test set (pos) ...
100% 49/49 [00:06<00:00, 7.67it/s]
testing pos samples ...
test pos sample accuracy: 0.8979591836734694
creating bow histograms for test set (neg) ...
100% 50/50 [00:06<00:00, 7.70it/s]
testing neg samples ...
test neg sample accuracy: 0.92
(ex4_cuda) PS D:\2023HS\Computer Vision\Object Recognition\exercise4_object_recognition_code>

```

3 CNN-based Classifier (40%)

3.1 A Simplified version of VGG Network (20%)

```
self.conv_block1 = nn.Sequential(nn.Conv2d(3, 64, 3, padding=1),
nn.ReLU(), nn.MaxPool2d(2, stride=2))
self.conv_block2 = nn.Sequential(nn.Conv2d(64, 128, 3,
padding=1), nn.ReLU(), nn.MaxPool2d(2, stride=2))
self.conv_block3 = nn.Sequential(nn.Conv2d(128, 256, 3,
padding=1), nn.ReLU(), nn.MaxPool2d(2, stride=2))
self.conv_block4 = nn.Sequential(nn.Conv2d(256, 512, 3,
padding=1), nn.ReLU(), nn.MaxPool2d(2, stride=2))
self.conv_block5 = nn.Sequential(nn.Conv2d(512, 512, 3,
padding=1), nn.ReLU(), nn.MaxPool2d(2, stride=2))
self.classifier = nn.Sequential(nn.Linear(self.fc_layer, 256),
nn.ReLU(), nn.Dropout(), nn.Linear(in features=256, out features=10))
```

```
def forward(self, x):
    """
    :param x: input image batch tensor, [bs, 3, 32, 32]
    :return: score: predicted score for each class (10 classes in
total), [bs, 10]
    """

    score = None
    # todo
    x = self.conv_block1(x)
    x = self.conv_block2(x)
    x = self.conv_block3(x)
    x = self.conv_block4(x)
    x = self.conv_block5(x)

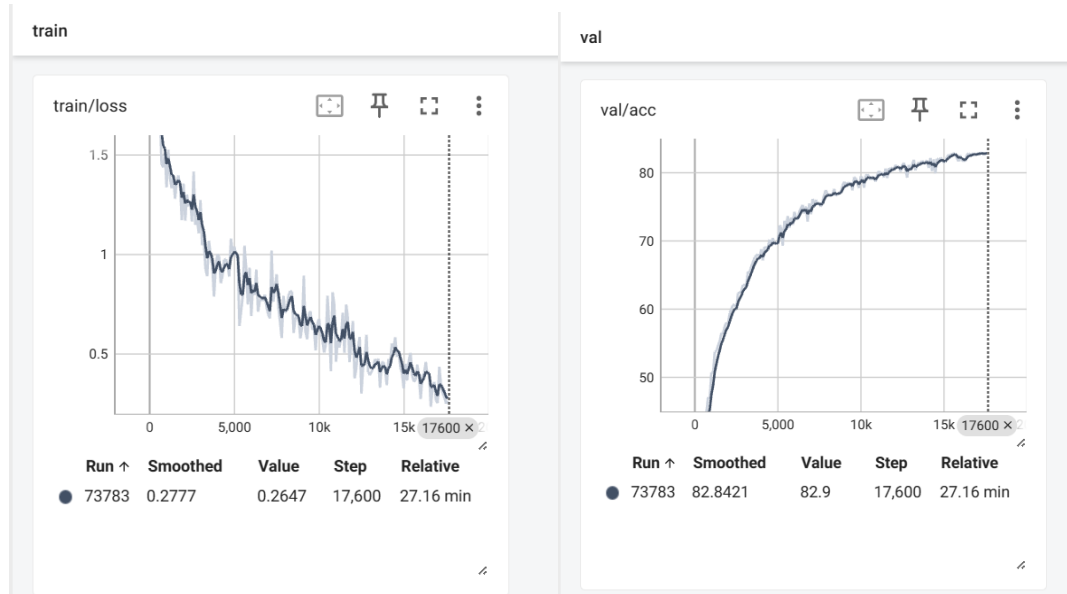
    #use torch.squeeze to turn [bs, 512, 1, 1] to [bs, 512]
    score = self.classifier(torch.squeeze(x))
```

```
return score
```

And the forward() function. Not much to say for this part.

3.2 Training and Testing (20%)

3.2.1 Training (10%)



I trained for 50 epochs with loss: 0.2647 and acc: 82.9.

3.2.2 Testing (10%)

```
Anaconda Powershell Prompt (Miniconda)
(ex4_cuda) PS D:\2023HS\Computer Vision\Object Recognition\exercise4_object_recognition_code> python test_cifar10_vgg.py
[INFO] test set loaded, 10000 samples in total.
79it [00:04, 19.52it/s]
test accuracy: 81.75
(ex4_cuda) PS D:\2023HS\Computer Vision\Object Recognition\exercise4_object_recognition_code>
```

The test accuracy is 81.75.